Da degradação do software aos Padrões de Projeto

Prof. Ricardo Vilela

Introdução

- Problemas recorrentes em sistemas orientados a objetos:
  - crescimento descontrolado de condicionais
  - acoplamento elevado
  - baixa extensibilidade
- Consequência direta:
  - aumento do custo de manutenção
  - maior risco de regressão
  - dificuldade de evolução do sistema
- Abordagem da aula:
  - identificação de **Code Smells**
  - compreensão de **Anti-Patterns**
  - aplicação de **Refatoração**
  - generalização para **Padrões de Projeto**
- Premissa central:
  - padrões não são ponto de partida
  - padrões emergem da resolução sistemática de problemas recorrentes

Problema motivador: Lógica baseada em tipo

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        }
    } else if (type.equals(anObject: "FOOD")) {
        price = quantity * 20;
    } else if (type.equals(anObject: "CLOTHING")) {
        price = quantity * 50;
        if (quantity > 5) {
            price -= 20;
        }
    }

    return price;
}
```

Problemas estruturais

- Acoplamento a String
- Violação do Open/Closed
- Baixa coesão
- Ausência de polimorfismo

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        }
    } else if (type.equals(anObject: "FOOD")) {
        price = quantity * 20;
    } else if (type.equals(anObject: "CLOTHING")) {
        price = quantity * 50;
        if (quantity > 5) {
            price -= 20;
        }
    }

    return price;
}
```

Impacto no sistema

- Baixa extensibilidade
- Alto custo de manutenção
- Risco de regressão
- Dificuldade de reutilização

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        }
    } else if (type.equals(anObject: "FOOD")) {
        price = quantity * 20;
    } else if (type.equals(anObject: "CLOTHING")) {
        price = quantity * 50;
        if (quantity > 5) {
            price -= 20;
        }
    }

    return price;
}
```

Natureza do problema

- Variação de comportamento baseada em tipo
- Uso de condicionais para decisão
- Falha na modelagem orientada a objetos

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        }
    } else if (type.equals(anObject: "FOOD")) {
        price = quantity * 20;
    } else if (type.equals(anObject: "CLOTHING")) {
        price = quantity * 50;
        if (quantity > 5) {
            price -= 20;
        }
    }

    return price;
}
```

O problema nem sempre é visível

- Nem todo problema aparece como erro
  - O Sistema funciona
  - Nenhum erro é exibido
  - Tudo parece “OK”

  - Mas o código começa a incomodar!!!

Um padrão de problema recorrente

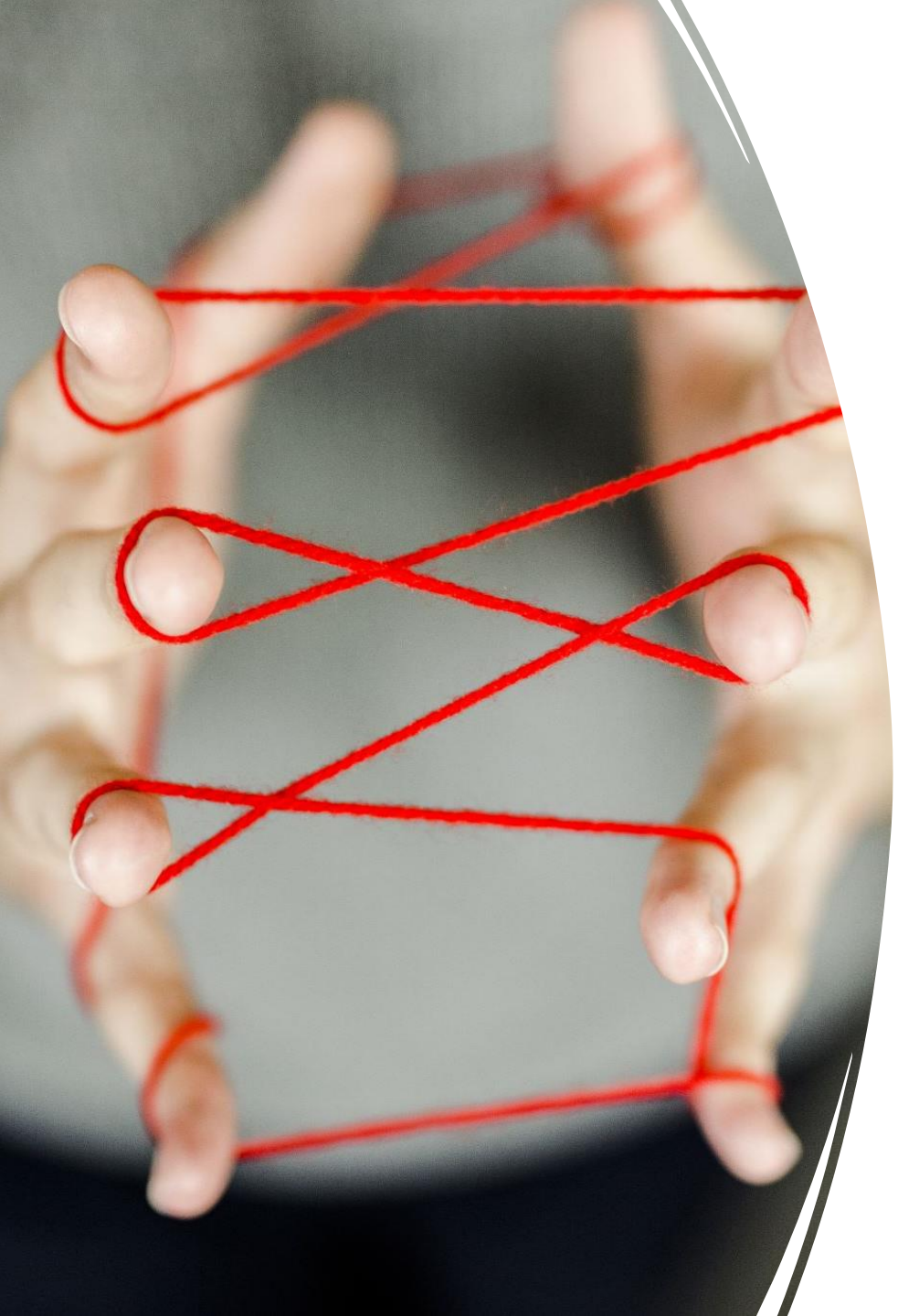
Problemas como o anterior não são casos isolados

Estruturas semelhantes aparecem com frequência

Características:

- código funcional
- dificuldade de evolução
- aumento de complexidade ao longo do tempo

Indício de um problema estrutural recorrente



Consequências desse tipo de estrutura

- Não são apenas questões estéticas
- Impactam diretamente a evolução do sistema
- Dificuldade de compreensão
- Crescimento progressivo da complexidade
- Alterações simples exigem mudanças em vários pontos
- Efeitos práticos:
  - Aumento do risco de erro
  - Maior esforço para manutenção
  - Menor previsibilidade do comportamento
- Estruturas assim tendem a degradar ao longo do tempo

Estrutura recorrente de decisão por tipo

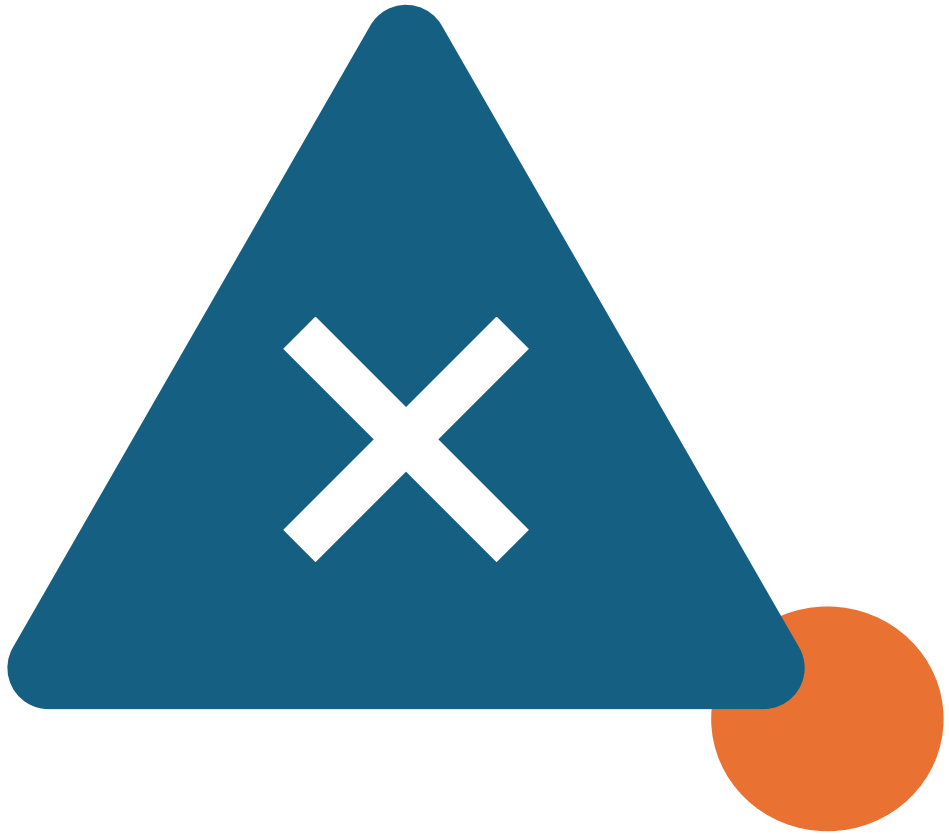
- Seleção de comportamento baseada em tipo
- Crescimento proporcional ao número de variações
- Características:
  - lógica centralizada
  - dependência de valores primitivos
  - ausência de encapsulamento
- Consequência estrutural:
  - dificuldade de extensão
  - necessidade de modificação contínua
- Estrutura recorrente em sistemas que não modelam variação adequadamente

```
if (type.equals(anObject: "ELECTRONIC")) { ... }
else if (type.equals(anObject: "FOOD")) { ... }
```

Nomeando o problema

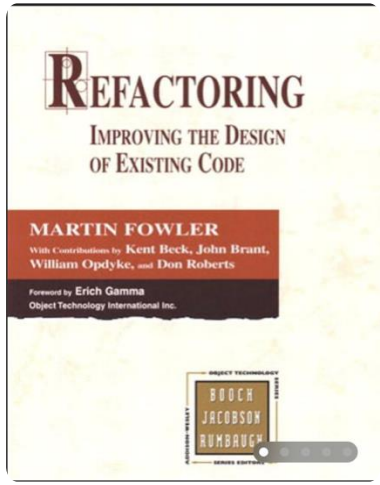
- Problemas como o anterior não são casos isolados
- Ocorrem de forma recorrente em sistemas reais
- Característica comum:
  - não impedem execução
  - dificultam evolução
- Indicam degradação estrutural do código

- Esses indícios recebem um nome na Engenharia de Software



Code Smells

- Termo introduzido por Martin Fowler
- Definição:
  - Indícios de degradação estrutural no código
  - Sintomas de problemas mais profundos de design
- Propriedades:
  - Não representam erros de execução
  - Não possuem definição estritamente formal
  - Dependem de interpretação contextual
- Papel no desenvolvimento:
  - Orientam a identificação de problemas de design
  - Indicam pontos candidatos à refatoração



Code Smell: Switch Statements

- Uso de estruturas condicionais para controlar variação de comportamento
- Crescimento proporcional ao número de casos
- Indícios de problema:
  - lógica de decisão centralizada
  - ausência de encapsulamento
  - dependência de valores primitivos
- Consequências:
  - Necessidade de modificação para inclusão de novos casos
  - Tendência à violação do Open/Closed

```
if (type.equals(anObject: "ELECTRONIC")) { ... }
else if (type.equals(anObject: "FOOD")) { ... }
else if (type.equals(anObject: "CLOTHING")) { ... }
```

Code Smell: Long Method

- Método concentra múltiplas responsabilidades
- Mistura regras de diferentes contextos (tipos de produto)
- Indícios de problema:
  - Aumento do tamanho com novas regras
  - Dificuldade de leitura e compreensão
  - Baixa granularidade das operações
- Consequências:
  - Dificuldade de manutenção
  - Baixa reutilização de partes do código
  - Maior probabilidade de erro em alterações

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) { ... }
    else if (type.equals(anObject: "FOOD")) { ... }
    else if (type.equals(anObject: "CLOTHING")) { ... }

    return price;
}
```

Code Smell: Primitive Obsession

- Uso de tipos primitivos para representar conceitos de domínio
- **String utilizada para modelar tipos de produto**
- Indícios de problema:
  - Ausência de abstração de domínio
  - Comportamento não associado aos dados
  - Validação dispersa ou inexistente
- Consequências:
  - Maior propensão a erros (ex: valores inválidos)
  - Dificuldade de evolução do modelo
  - Duplicação de lógica associada ao tipo

```
public double calculatePrice(String type, int quantity) {
```

Múltiplos Code Smells em uma única estrutura

- Presença simultânea de múltiplos smells:
  - Switch Statements → variação controlada por condicionais
  - Long Method → concentração de responsabilidades
  - Primitive Obsession → uso de String para representar domínio

```
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) { ... }
    else if (type.equals(anObject: "FOOD")) { ... }
    else if (type.equals(anObject: "CLOTHING")) { ... }

    return price;
}
```

Múltiplos Code Smells em uma única estrutura

- Intepretação:
  - Smells raramente aparecem isolados
  - Tendem a se combinar em estruturas recorrentes
- Implicação:
  - Problemas de design são sistêmicos, não pontuais

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) { ... }
    else if (type.equals(anObject: "FOOD")) { ... }
    else if (type.equals(anObject: "CLOTHING")) { ... }

    return price;
}
```

Evolução dos problemas de design

- Code smells indicam degradação local no código
- Quando persistem, tendem a se acumular
- Consequência:
  - Aumento da complexidade estrutural
  - Perda de controle sobre o design
- Problemas deixam de ser pontuais e passam a ser estruturais

Anti-Patterns

- Definição:
  - Soluções recorrentes que produzem efeitos negativos
  - Práticas adotadas que agravam o problema ao longo do tempo
- Características:
  - Parecem resolver o problema inicialmente
  - Introduzem novas dificuldades estruturais
  - Tornam o sistema progressivamente mais difícil de evoluir
- Conclusão:
  - Representam a consolidação de más decisões de design
  - Anti-patterns são a evolução natural de problemas não tratados

Anti-Pattern: God Class

Classe concentra múltiplas responsabilidades

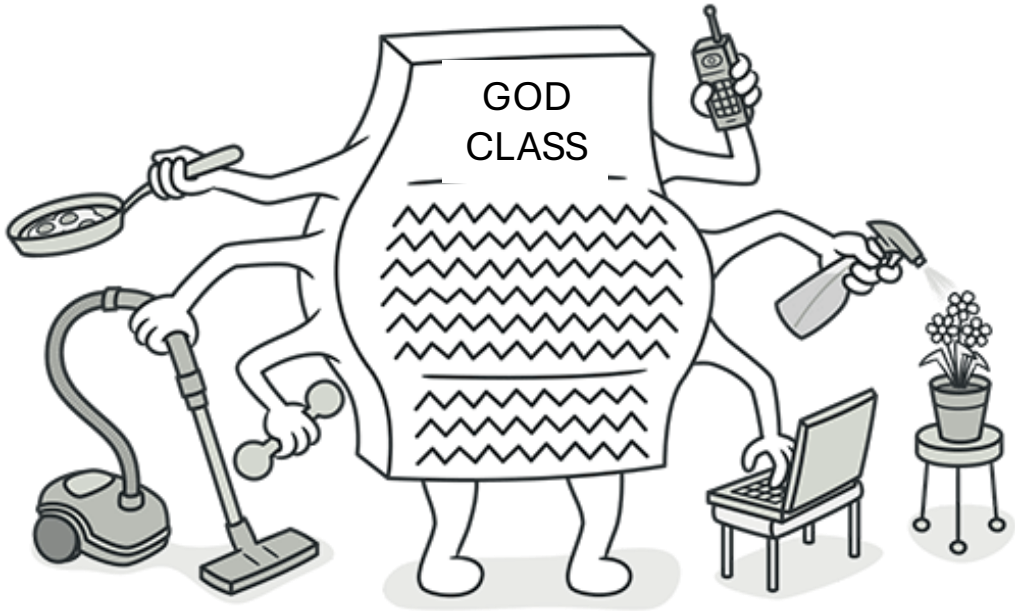
Atua como ponto central de diversas operações

Indícios:

- Crescimento contínuo da classe
- Acúmulo de regras de negócio distintas
- Alta dependência por outras partes do sistema

Consequências:

- Baixa coesão
- Alto acoplamento
- Dificuldade de manutenção e evolução



Anti-Pattern: Copy-Paste Programming

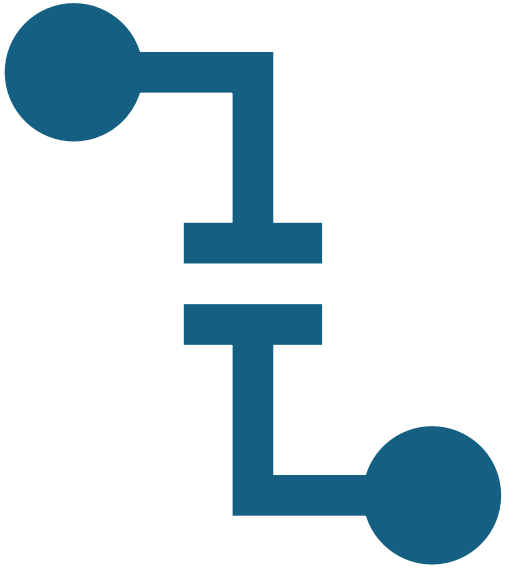
- Repetição de lógica com pequenas variações
- Código duplicado distribuído pelo sistema
- Indícios:
  - trechos semelhantes em múltiplos pontos
  - alterações exigem mudanças repetidas
  - inconsistências entre versões do mesmo código
- Consequências:
  - aumento do custo de manutenção
  - maior risco de inconsistência
  - propagação de erros

```
if (type.equals(anObject: "ELECTRONIC")) {
    price = quantity * 100;
    if (quantity > 10) price *= 0.9;
}

if (type.equals(anObject: "BOOK")) {
    price = quantity * 100;
    if (quantity > 10) price *= 0.9;
}
```

Anti-Pattern: Spaghetti Code

- Estrutura de código sem organização clara
- Fluxo de execução difícil de rastrear
- Indícios:
  - múltiplas ramificações e dependências implícitas
  - ausência de separação de responsabilidades
  - forte acoplamento entre partes do sistema
- Características:
  - dificuldade de leitura
  - baixa previsibilidade de comportamento
  - alta complexidade acidental
- Consequências:
  - manutenção onerosa
  - elevada probabilidade de regressão
  - evolução do sistema torna-se arriscada



```java
public void processOrder(String type, int quantity) {

    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        } else {
            if (quantity == 5) {
                price -= 10;
            }
        }
    } else {
        if (type.equals(anObject: "FOOD")) {
            price = quantity * 20;
        } else {
            if (type.equals(anObject: "CLOTHING")) {
                price = quantity * 50;
                if (quantity > 5) {
                    price -= 20;
                }
            }
        }
    }

    if (price > 500) {
        if (type.equals(anObject: "ELECTRONIC")) {
            System.out.println(x: "High value electronic");
        } else {
            System.out.println(x: "High value order");
        }
    }

    System.out.println(price);
}
```

Code Smells vs. Anti-Patterns

- Code Smells
  - indícios locais de degradação no código
  - associados a trechos específicos
  - sinalizam necessidade de refatoração
- Anti-Patterns
  - soluções recorrentes com efeitos negativos
  - manifestam-se no nível estrutural do sistema
  - resultam da acumulação de más decisões
- Relação entre os conceitos:
  - smells podem evoluir para anti-patterns
  - anti-patterns emergem quando smells não são tratados
- Interpretação:
  - smells → sintomas
  - anti-patterns → manifestação estrutural do problema

Introdução à Refatoração

- Conceito formalizado por Martin Fowler
- Definição:
  - processo disciplinado de reestruturação do código-fonte
  - preservando o comportamento observável do sistema
- Objetivos:
  - melhoria da qualidade interna do software
  - aumento da legibilidade, manutenibilidade e extensibilidade
- Características:
  - realizada de forma incremental e sistemática
  - orientada pela identificação de problemas de design (code smells)
- Papel no desenvolvimento:
  - mecanismo contínuo de evolução do design
  - instrumento para prevenção da consolidação de anti-patterns

Invariância Comportamental

Princípio central da refatoração:

- A estrutura interna do código é modificada
- O comportamento observável permanece inalterado

Implicação:

- Mudanças não devem afetar a saída do sistema
- Interfaces e contratos externos são preservados

Requisito fundamental:

- Existência de mecanismos de verificação (testes)
- Garantia de que a refatoração não introduz defeitos

Interpretação:

- Refatorar não é adicionar funcionalidade
- Refatorar é melhorar a forma, mantendo a função

Refatoração: Extract Method (antes)

- Método concentra múltiplas regras de negócio
- Baixa decomposição das responsabilidades
- Problemas observáveis:
  - dificuldade de leitura
  - lógica não modularizada
  - reutilização inexistente

```java
public double calculatePrice(String type, int quantity) {
    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        }
    } else if (type.equals(anObject: "FOOD")) {
        price = quantity * 20;
    } else if (type.equals(anObject: "CLOTHING")) {
        price = quantity * 50;
        if (quantity > 5) {
            price -= 20;
        }
    }

    return price;
}
```

Refatoração: Extract Method (depois)

```java
public double calculatePrice(String type, int quantity) {

    if (type.equals(anObject: "ELECTRONIC")) {
        return calculateElectronic(quantity);
    } else if (type.equals(anObject: "FOOD")) {
        return calculateFood(quantity);
    } else if (type.equals(anObject: "CLOTHING")) {
        return calculateClothing(quantity);
    }

    return 0;
}
```

```java
private double calculateElectronic(int quantity) {
    double price = quantity * 100;
    if (quantity > 10) {
        price *= 0.9;
    }
    return price;
}
private double calculateFood(int quantity) {
    return quantity * 20;
}

private double calculateClothing(int quantity) {
    double price = quantity * 50;
    if (quantity > 5) {
        price -= 20;
    }
    return price;
}
```

Refatoração: Extract Method (depois)

Separação de responsabilidades por operação

Redução da complexidade do método principal

Ganhos imediatos:

- melhoria da legibilidade
- maior granularidade das regras
- possibilidade de reutilização

Limitação:

- decisão por tipo ainda centralizada (**if/else**)

Limitações da refatoração aplicada

Persistência da decisão por tipo
lógica ainda controlada por if/else

Violação do princípio Open/Closed
inclusão de novos tipos exige modificação do método

Acoplamento a representação primitiva
dependência de String para definição de comportamento

Centralização da lógica de decisão
conhecimento de todos os tipos concentrado em um único ponto

Substituição de condicionais por polimorfismo

- Direção da refatoração:
  - encapsular comportamento em objetos
  - eliminar decisão baseada em tipo
- Mudança estrutural:
  - cada tipo passa a ser responsável pelo seu comportamento
- Consequência:
  - remoção progressiva do if/else
  - distribuição da lógica entre classes

```
interface Product {
    double calculatePrice(int quantity);
}
```

Implementações concretas

```java
class Electronic implements Product {
    public double calculatePrice(int quantity) {
        double price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        }
        return price;
    }
}
```

```java
class Food implements Product {
    public double calculatePrice(int quantity) {
        return quantity * 20;
    }
}
```

```java
class Clothing implements Product {
    public double calculatePrice(int quantity) {
        double price = quantity * 50;
        if (quantity > 5) {
            price -= 20;
        }
        return price;
    }
}
```

Uso da nova estrutura (sem condicionais)

```
class OrderService {

    public double calculatePrice(Product product, int quantity) {
        return product.calculatePrice(quantity);
    }
}
```

```java
public void processOrder(String type, int quantity) {

    double price = 0;

    if (type.equals(anObject: "ELECTRONIC")) {
        price = quantity * 100;
        if (quantity > 10) {
            price *= 0.9;
        } else {
            if (quantity == 5) {
                price -= 10;
            }
        }
    } else {
        if (type.equals(anObject: "FOOD")) {
            price = quantity * 20;
        } else {
            if (type.equals(anObject: "CLOTHING")) {
                price = quantity * 50;
                if (quantity > 5) {
                    price -= 20;
                }
            }
        }
    }

    if (price > 500) {
        if (type.equals(anObject: "ELECTRONIC")) {
            System.out.println(x: "High value electronic");
        } else {
            System.out.println(x: "High value order");
        }
    }

    System.out.println(price);
}
```

Antes....

Insight: Recorrência em problemas e soluções

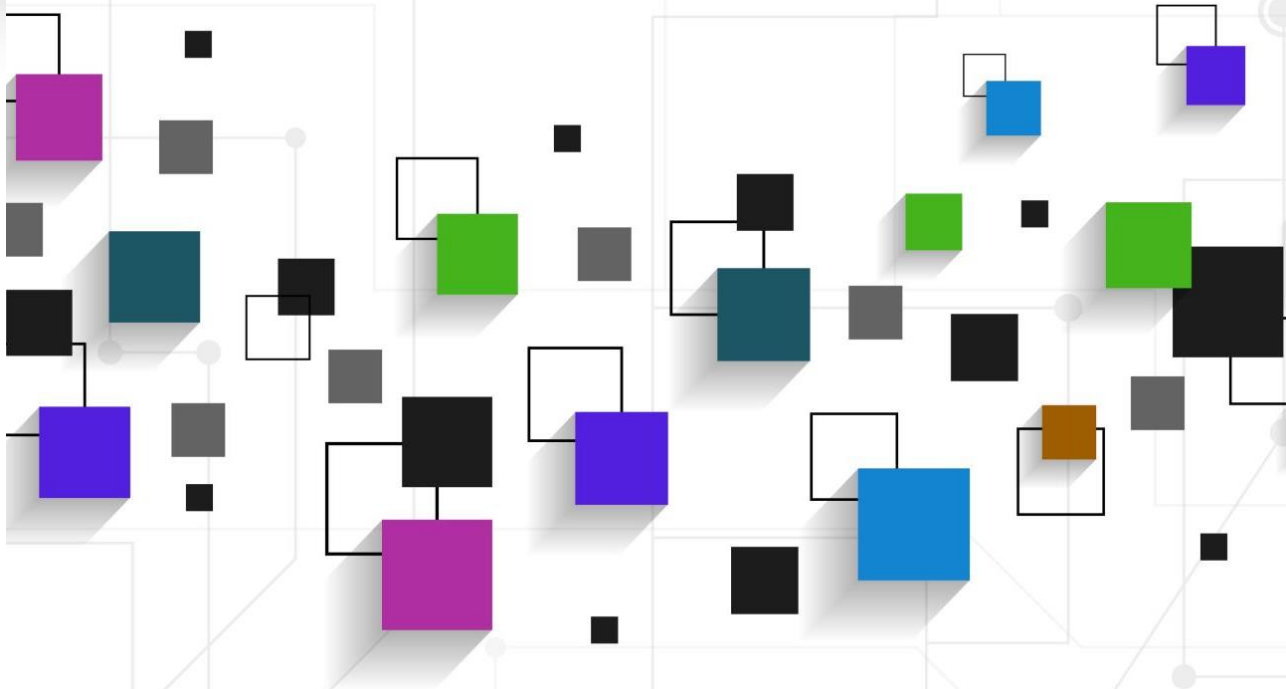
- Problemas observados:
  - Variação de comportamento
  - Uso de condicionais
  - Centralização de lógica
- Observação:
  - Esse tipo de problema aparece repetidamente em diferentes sistemas
- Consequência:
  - Soluções semelhantes são aplicadas de forma recorrente
  - Desenvolvedores experientes reconhecem esses padrões
- Interpretação:
  - A solução não é pontual
  - É uma resposta recorrente a um tipo de problema

Design Patterns

- Definição:
  - Soluções recorrentes para problemas recorrentes de design
- Características:
  - Baseadas em experiência acumulada
  - Independentes de linguagem específica
  - Aplicáveis em diferentes contextos
- Importância:
  - Evitam a reinvenção de soluções
  - Orientam decisões de design
  - Promovem reuso de conhecimento

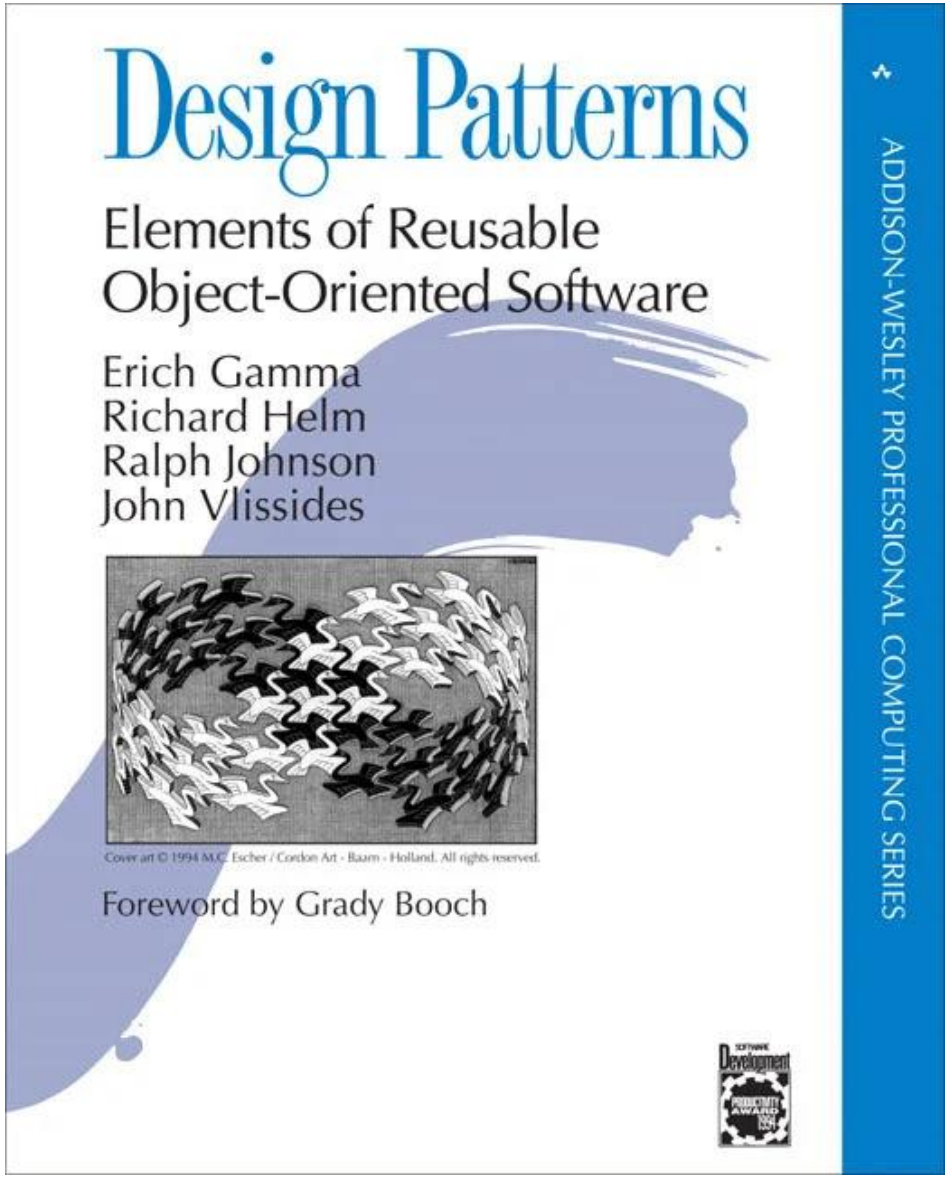
Estrutura de um Design Pattern

- **Problema**
  - Situação recorrente de design
  - Contexto em que o padrão é aplicável
- **Solução**
  - Estrutura de classes e objetos
  - Distribuição de responsabilidades
- **Contexto**
  - Condições nas quais a solução é adequada
- **Consequências**
  - Trade-offs da aplicação do padrão
  - Impacto em flexibilidade, reutilização e complexidade



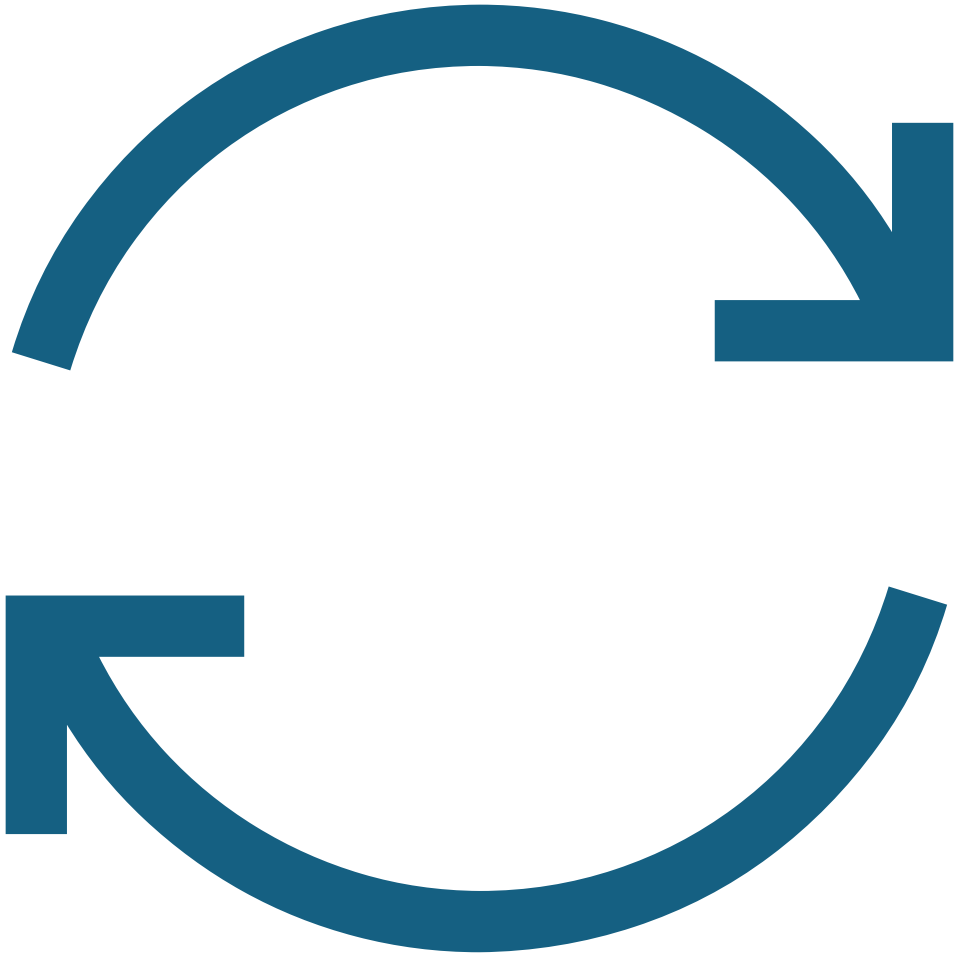
Categorias de Design Patterns (GoF)

- Os padrões do catálogo GoF são organizados em três categorias principais:
  - Criacionais
    - Tratam da criação de objetos
    - Abstraem o processo de instanciamento
  - Estruturais
    - Tratam da composição de classes e objetos
    - Definem como estruturas são organizadas
  - Comportamentais
    - Tratam da interação entre objetos
    - Definem como responsabilidades são distribuídas



Síntese da aula

- Código funcional ≠ bom design
- Problemas recorrentes:
  - Code Smells → Anti-Patterns
- Intervenção:
  - Refatoração
- Generalização:
  - Design Patterns



Da degradação do software aos Padrões de Projeto

Prof. Ricardo Vilela